

university of groningen / faculty of science and engineering / artificial intelligence computational cognitive science

Model Predictive Control for Trajectory Tracking on Crazyflie 2.1 Quadrotor

Nicolas Cheptine Smith Samuel Preston Jones

October 12, 2026

Integration Project Report
Faculty of Science and Engineering
University of Groningen, The Netherlands

1 Introduction

Unmanned aerial vehicles (UAVs), and quadrotors in particular, have attracted growing interest across research and industry due to their versatility in applications such as infrastructure inspection, precision agriculture, search and rescue, and autonomous delivery. A fundamental requirement for automating such tasks is the ability to reliably guide a UAV along a desired route while respecting physical constraints and adapting to real-world disturbances. For nano-quadrotors operating in structured environments, such as indoor inspection or swarm coordination, this demands controllers that are both constraint-aware and capable of tracking dynamically feasible references with high spatial accuracy.

Quadrotor drones are, in principle, a simple system since they only have four actuators and no moving limbs or extensions. However, while the design itself is simple, it must be able to freely translate and rotate in a 3D space, i.e. it has three translational and three rotational degrees of freedom. The system has a total of six degrees of freedom, but four motors to actuate. Thus, the system is what is known as underactuated. This underactuation leads to coupled dynamics. For instance, to translate laterally, the vehicle must first rotate (roll) to tilt its total thrust vector, creating a horizontal force component while simultaneously adjusting thrust to maintain altitude. This inherent coupling between translational and rotational motion is the central difficulty in quadrotor control.

Model Predictive Control (MPC) offers a principled framework for this challenge by repeatedly solving a constrained optimization problem over a receding time horizon. Unlike classical PID-based approaches, MPC explicitly accounts for system dynamics, anticipates future behavior, and enforces state and input constraints directly within the control law. This makes it particularly well-suited to nano-quadrotors such as the Crazyflie 2.1, where actuator limits, attitude constraints, and fast translational dynamics all interact.

This project addresses the design, implementation, and experimental validation of an MPC-based trajectory tracking controller for the Crazyflie 2.1 nano-quadrotor. Trajectory tracking requires the vehicle to follow a time-stamped sequence of reference states, a formulation directly applicable to scenarios where precise spatial and temporal coordination is required. We adopt a cascaded control architecture in which the MPC outer loop generates attitude and thrust commands, delegating fast attitude stabilization to the quadrotor's onboard PID firmware. This separation makes real-time off-board MPC computationally feasible without sacrificing closed-loop performance.



Figure 1: Crazyflie 2.1 quadrotor drone (Bitcraze, n.d.).

1.1 Background

To facilitate discussion, standard quadrotor terminology is established here and used throughout this work. Vehicle orientation is described in a body-fixed coordinate frame using three Euler angles, Roll (ϕ), Pitch (θ), and Yaw (ψ) angles—collectively known as RPY, following the aerospace convention illustrated in Figure 2:

- Roll (ϕ): Rotation about the X-axis (longitudinal axis). Positive roll causes the drone to tilt to the right (clockwise when viewed from behind), generating lateral thrust.
- Pitch (θ): Rotation about the Y-axis (lateral axis). Positive pitch tilts the drone forward, producing forward/backward thrust.
- Yaw (ψ): Rotation about the Z-axis (vertical axis). Positive yaw rotates the drone clockwise when viewed from above (nose turns right).

In practice, however, quaternions are used for numerical computation within the mathematical models and controllers due to their advantages, which are discussed at the end of this section.

For quadrotors, the body-fixed frame is typically defined with

- X-axis: Forward direction (toward ”front” of drone)
- Y-axis: Right direction
- Z-axis: Downward direction (following NED convention) or upward (following ENU convention). During this work, we will use ENU convention, so the z-axis is in the upward direction. This can be seen in Figure 2.

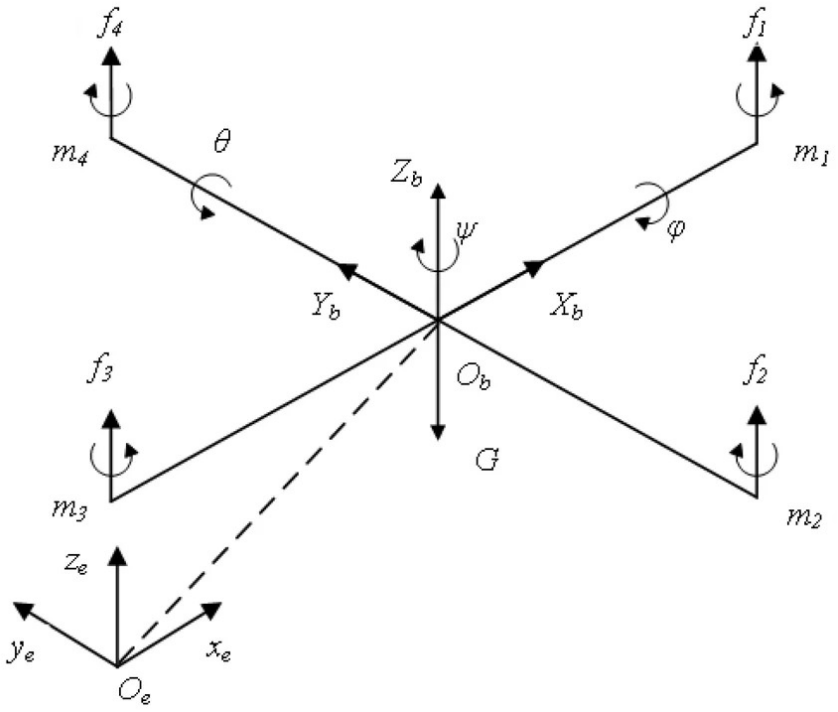


Figure 2: Roll, pitch, and yaw rotation axes on a quadrotor drone.

Quadrotors have 4 propellers, two of which rotate in one direction and two of which rotate in the opposite direction. In the Crazyflie 2.1 quadrotor, the propellers that are opposite each other, with respect to the center of the drone, rotate in the same direction. This configuration is necessary to avoid yaw angle drift. If all the propellers rotated in the same direction, it would generate a torque on the drone that would rotate it in the opposite direction to that of the propellers. Thus, by having two propellers rotating in opposite directions to the other two, this torque is nullified.

2 Literature Survey

Quadrotor control has been approached through a range of methods of increasing sophistication. Classical PID controllers remain widely deployed due to their simplicity and low computational cost, and form the basis of most commercial firmware, including that of the Crazyflie platform (Bitcraze AB, n.d.). However, PID controllers do not handle constraints directly, offer limited predictive capability, and require careful manual tuning when operating conditions change. Linear controllers such as LQR improve on this by providing systematic gain design but similarly lack constraint handling and rely on linearization around a fixed operating point, which considerably reduces robustness.

MPC addresses these shortcomings by formulating control as a constrained optimization problem solved repeatedly over a receding horizon. This allows the controller to anticipate future behavior, explicitly respect state and input constraints, handle nonlinear dynamics through appropriate problem formulations, and robustly adapt to disturbances by continuously re-solving the optimization problem. MPC has been successfully applied to quadrotor control across a range of platforms and flight regimes (Kamel, Stastny,

Alexis, & Siegwart, 2017). A key practical consideration is computational tractability: solving a nonlinear optimal control problem at the control frequency of a quadrotor demands efficient solvers and problem formulations.

In autonomous quadrotor design, two primary control paradigms exist: Integrated Guidance and Control (IGC) and Separated Guidance and Control (SGC). IGC formulates a single controller that commands motor Pulse Width Modulation (PWM) signals directly, capturing the full nonlinear dynamics but at high computational cost. SGC instead delegates fast inner-loop attitude stabilization to the onboard firmware, with the MPC outer loop generating attitude and thrust commands. For a Crazyflie nano-quadrotor, where attitude control already runs at 500 Hz (Bitcraze AB, n.d.), the SGC approach is well motivated: it reduces the dimensionality of the optimization problem, enables off-board computation without sacrificing attitude stability, and leverages the well-tuned firmware attitude controller. This is the architecture adopted in this project, inspired by the work of (Leprich, Rosenfelder, Hermle, Chen, & Eberhard, 2026).

A further modelling choice concerns how the closed-loop attitude dynamics are represented in the MPC prediction model. When the inner-loop attitude controller operates at sufficiently high bandwidth, the closed-loop roll and pitch responses can be approximated as first-order systems with experimentally identified time constants (Kamel et al., 2017; Llanes, Kakish, Williams, & Coogan, 2024). This approximation yields an efficient almost-linear MPC formulation, and it is standard practice for systems with high-frequency inner-loop attitude controllers.

Within the SGC framework, trajectory tracking MPC is well-established (Kamel et al., 2017). The controller receives a precomputed time-parameterized reference and minimizes tracking error subject to the prediction model and constraints. This formulation is directly applicable to structured indoor tasks, such as inspection, coordination in swarms, or navigation through constrained environments, where precise temporal and spatial adherence to a reference is required.

3 Problem Formulation

3.1 Quadrotor Dynamics

The translational dynamics follow from Newton's equations:

$$\ddot{\xi} = -\begin{bmatrix}0\\0\\g\end{bmatrix} + \frac{1}{m}R_{IB}(\eta)\begin{bmatrix}0\\0\\\Delta T + mg\end{bmatrix} \tag{1}$$

with mass m, gravitational acceleration g, differential thrust $\Delta T = T - mg$, and rotation matrix $R_{IB}(\eta)$ mapping body-frame thrust to inertial-frame acceleration.

The full rotational dynamics are described by

$$\dot{\zeta} = \frac{1}{2}\zeta \otimes \Omega, \quad \dot{\Omega} = J^{-1}(\mu + \mu_{ext} - \Omega \times J\Omega),$$

where ζ is the attitude quaternion, Ω the angular velocity, μ the control moment, μ_{ext} the external disturbance moment and J the drone's inertia tensor. The operator $\otimes$ represents the Hamiltonian product and $\times$ the cross-product. However, this model is quite complex and computationally expensive for the solver; thus, it was substituted by the following simplified model.

The Crazyflie 2.1 is modeled using a Separated Guidance and Control (SGC) architecture, in which the MPC outer loop generates high-level commands consumed by the onboard PID attitude controller. Since the attitude controller operates at 500 Hz (Bitcraze AB, n.d.), the closed-loop roll and pitch responses are well-approximated by first-order dynamics (Kamel et al., 2017; Llanes et al., 2024)

$$\dot{\phi} = \frac{1}{T_\phi}(\phi_{\text{cmd}} - \phi), \quad \dot{\theta} = \frac{1}{T_\theta}(\theta_{\text{cmd}} - \theta), \quad \dot{\psi} = \dot{\psi}_{\text{cmd}}. \tag{2}$$

The attitude time constants T_ϕ and T_θ were taken from the MPC implementation associated with the work by (Llanes et al., 2024) (the repo) which identifies them experimentally on the same platform. These constants correspond to the amount taken for the attitude setpoint to be tracked.

Thus, the system state is

$$x = [\xi^\top, \dot{\xi}^\top, \eta^\top]^\top \in \mathcal{X} \subset \mathbb{R}^9 \tag{3}$$

where $\xi = [x, y, z]^\top$ denotes position in the inertial frame and $\eta = [\phi, \theta, \psi]^\top$ represents the Euler angles (roll, pitch, yaw). The MPC control input is

$$u = [\Delta T, \phi_{\text{cmd}}, \theta_{\text{cmd}}, \dot{\psi}_{\text{cmd}}]^\top \in \mathcal{U} \subset \mathbb{R}^4. \tag{4}$$

Note that the firmware accepts roll and pitch angle commands alongside yaw-rate commands; yaw is treated as a rate rather than an angle to avoid wrap-around issues in the optimization.

3.2 Trajectory Tracking MPC

A reference trajectory

$$y_{\text{ref}}(t) = [x_{\text{ref}}(t), y_{\text{ref}}(t), z_{\text{ref}}(t), \dot{x}_{\text{ref}}(t), \dot{y}_{\text{ref}}(t), \dot{z}_{\text{ref}}(t), \phi_{\text{ref}}(t), \theta_{\text{ref}}(t), \psi_{\text{ref}}(t)]^\top$$

is provided a priori. At each time t, the MPC solves the Optimal Control Problem (OCP):

$$\min_{\bar{u}(\cdot\,|\,t)} \int_t^{t+T_p} \|x(\tau) - y_{\text{ref}}(\tau)\|_Q^2 + \|\bar{u}(\tau\,|\,t)\|_R^2 \, \mathrm{d}\tau \tag{5}$$

subject to

$$\begin{aligned}
&\dot{\bar{x}}(\tau\,|\,t) = f(\bar{x}(\tau\,|\,t), \bar{u}(\tau\,|\,t)), \quad \forall \tau \in [t, t+T_p] \\
&\bar{x}(\tau\,|\,t) \in \mathcal{X}, \quad \bar{u}(\tau\,|\,t) \in \mathcal{U}, \quad \forall \tau \in [t, t+T_p] \\
&\bar{x}(t\,|\,t) = x(t)
\end{aligned}$$

where Q and R are positive definite weighting matrices, $\mathcal{X}$ and $\mathcal{U}$ encode state and input constraints, and T_p is the prediction horizon.

The weighting matrices Q and R can be tuned empirically. Different parts of the state and the input can be penalized more or less, allowing for a high degree of customization in the desired performance.

3.3 Trajectory Generation

To evaluate the performance of the proposed control architecture, two reference trajectories were defined as smooth, parametric curves in three-dimensional space: a helix and a lemniscate.

The helical trajectory is parameterized as:

$$x(t) = \alpha \cos(t) - \alpha, \quad y(t) = \beta \sin(t), \quad z(t) = \frac{t}{2\pi} \tag{6}$$

where $t \in [0, 2\pi]$, α is the x-axis size parameter, β the y-axis size parameter. This trajectory prescribes a single ascending revolution, combining uniform circular motion in the horizontal plane with a constant rate of climb in the vertical axis. It is commonly used to assess a controller's ability to simultaneously track coupled translational dynamics across all three axes.

The lemniscate trajectory is parameterized as:

$$x(t) = \alpha \cos\left(t + \frac{\pi}{2}\right), \quad y(t) = \beta \sin\left(2\left(t + \frac{\pi}{2}\right)\right), \quad z(t) = \gamma \tag{7}$$

where t, α, β same as before, and γ the altitude parameter. In this case, the drone first hovers to the desired altitude and the trajectory simply maintains that altitude. This planar trajectory traces a figure-eight pattern and serves as a benchmark for evaluating tracking performance under frequent changes in curvature and heading direction, placing particular demand on the rotational dynamics of the vehicle.

In both cases, n uniformly spaced waypoints were sampled along the parameter t to produce the reference position sequence supplied to the controller. The trajectories were designed to remain within a bounded spatial envelope, with a maximum radial extent of 0.5 m, making them suitable for evaluation in both simulation and constrained experimental environments.

The resulting waypoints are passed into the minimum-snap polynomial optimization framework of Richter et al. (Richter, Bry, & Roy, 2016). This method fits a sequence of piecewise polynomial segments through the prescribed waypoints by solving an unconstrained quadratic program that minimizes the snap (fourth derivative of position) of the resulting path. Minimizing snap is motivated by the fact that, for a quadrotor, snap is directly proportional to the required motor thrust and attitude accelerations, thereby producing smooth, dynamically efficient trajectories. The optimizer jointly solves for the polynomial coefficients across all segments simultaneously, enforcing continuity of position and its derivatives up to arbitrary order at the waypoint transitions. The output is a continuous, differentiable position trajectory along with its higher-order derivatives, which are supplied directly as reference signals to the controller.

4 Methods

4.1 Experimental Setup

The controller runs off-board on a laptop, communicating with the Crazyflie 2.1 firmware via a Crazyradio USB dongle using ROS2 and the Crazyswarm2 middleware (Preiss*, Hönig*, Sukhatme, & Ayanian, 2017). The onboard PID attitude controller receives ϕ_{cmd}, θ_{cmd}, ψ_{cmd}, and ΔT at the MPC control frequency.

State feedback is provided by a motion capture (MoCap) system comprising eight QualiSys Miqus 3 cameras, which supply high-rate pose measurements to the Crazyflie's onboard Extended Kalman filter (EKF). The EKF fuses MoCap position and orientation measurements with onboard IMU data (accelerometers and gyroscopes) to produce a full state estimate. This fused estimate is broadcast over the Crazyradio link and used as the MPC initial condition $x(t)$ at each control step.

Velocities are not directly available from MoCap and are estimated via two-point finite differencing of successive position measurements. Since MoCap delivers sub-centimeter accuracy at 100 Hz, finite differences provide reliable velocity estimates under standard lab conditions. However, occasional packet loss or timing jitter can produce anomalous velocity spikes. To tackle this issue, a sliding window outlier detector was implemented. It solves the problem by comparing each finite-difference estimate against the mean absolute deviation of the previous five estimates; if the current estimate deviates beyond a fixed threshold, it is replaced by the onboard log variables `stateEstimate.Vx`, `stateEstimate.Vy`, and `stateEstimate.Vz` streamed from the Crazyflie's own state estimator. This hybrid scheme provides robustness to transient MoCap dropouts without relying solely on the noisier onboard estimate and was found to give better performance than using either the finite difference or the onboard estimate alone.

4.2 Numerical Implementation

The OCP is solved using acados (Verschueren et al., 2021) with CasADi (Andersson, Gillis, Horn, Rawlings, & Diehl, 2018) used to define the explicit continuous-time dynamics f symbolically. The multiple shooting transcription is used: acados internally discretizes the continuous-time model over each shooting interval using an explicit Runge-Kutta integrator (ERK4), propagating the nonlinear dynamics while the OCP structure is exploited by the Real-Time Iteration scheme of the underlying Sequential Quadratic Programming solver. This avoids the need for manual discretization and ensures the prediction model remains consistent with the nonlinear dynamics throughout the horizon.

At each control step, the OCP is warm-started by shifting the previous solution forward by one step: $\bar{u}_i \leftarrow \bar{u}^\star_{i+1}$ and $\bar{x}_i \leftarrow \bar{x}^\star_{i+1}$, with the final step duplicated as a simple guess. The SQP solver performs up to 50 iterations per optimization. In practice, the warm start ensures the solver converges rapidly from the shifted prior solution, keeping solve times within the 10 ms control period.

4.3 Benchmark Controller

The Crazyflie firmware comes with a built-in Mellinger controller. This controller was used to benchmark the performance of the MPC controller. The trajectories and experiments performed on the MPC controller were also performed on the Mellinger for comparison.

The Mellinger controller is a geometric nonlinear controller for quadrotors, originally proposed by Mellinger and Kumar (Mellinger & Kumar, 2011) and implemented in the Crazyflie firmware as part of the Crazyswarm project. It exploits the differential flatness of quadrotor dynamics (the property that the full system state and inputs are algebraically determined by the flat outputs (x, y, z, ψ) and their derivatives) to map position and velocity errors directly into desired body orientation without small-angle approximations. The controller operates as two nested loops: an outer position loop at 100 Hz computes a desired thrust vector from position and velocity errors plus feedforward acceleration, from which a desired rotation matrix is constructed given the desired yaw; an inner attitude loop at 250 Hz then computes an attitude error and applies proportional-derivative feedback on this error and the angular velocity error to produce torque commands.

4.4 Thrust Calibration

The Crazyflie firmware accepts thrust commands via the Crazyswarm2 function `cmdVel()` as dimensionless Pulse Width Modulation (PWM) values in the range $[0, 6000]$, whereas the MPC formulation requires thrust in Newtons as a physical input. A calibration experiment was conducted to establish the mapping between these two representations. The Crazyflie was mounted on an improvised thrust stand: the drone was fixed atop a water bottle resting on a digital scale, oriented so that upward thrust reduced the apparent weight registered by the scale. The experimental setup is shown in Figure 3. A sequence of PWM values spanning the operational range was commanded via `cmdVel()`, and the corresponding thrust in grams was computed from the change in scale reading. This method was inspired by a similar experiment discussed in the documentation (Bitcraze AB, 2026). They found a quadratic relationship between PWM and thrust in grams. The data collected suggested, which can be seen in Figure 4, a linear relationship. A linear



Figure 3: Setup used to obtain the linear fit between PWM command and thrust output in grams.

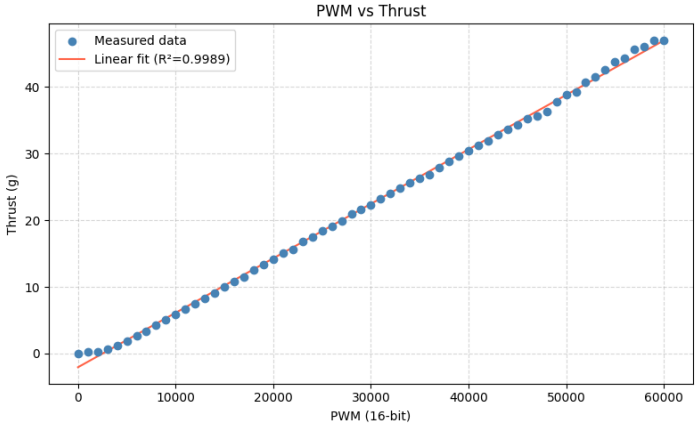


Figure 4: Linear fit between PWM commands and thrust output in grams.

fit achieved $R^2 = 0.9989$, confirming a highly linear relationship across the operating range. The inverse of this mapping is used at each control step to convert the MPC thrust output in Newtons to the PWM value sent to the firmware.

It is important to note that the improvised thrust stand is not the ideal apparatus for these measurements. While the drone was mounted as rigidly as possible to the bottle, it is difficult to guarantee that small vibrations or motions of the airframe did not interfere with the scale readings. Nevertheless, the high $R^2 = 0.9989$ suggests these effects were minor across the operational range. As can be seen in Figure 4, thrust remains at zero for low PWM values. This is likely due to the motors being unable to overcome static friction at low PWM values, resulting in no thrust being generated. This deadband behavior is not captured by the linear fit and should be borne in mind when commanding low thrust values. It is also important to note that the diameter of the scale platform, coinciding with the diameter of the bottle used as a stand, is not a coincidence but a design choice. This is important to ensure that the displaced air from the rotors does not fall on the weighing platform since this adds apparent weight and renders the thrust measured in grams incorrect.

The Crazyswarm2 function `cmdVel()` accepts roll, pitch, yaw rate, and thrust commands, forwarding them to the onboard attitude controller. Thrust is specified in dimensionless 16-bit PWM units, spanning the range $[0, 2^{16} - 1] = [0, 65535]$. In practice, `cmdVel()` accepts thrust values up to 6000 rather than the full 16-bit maximum. This headroom is intentional; the onboard stabilizer requires authority to increase thrust above the commanded value in order to correct attitude disturbances, which would be impossible if the commanded thrust were already at the hardware limit.

5 Results & Discussion

5.1 MPC Parameters

The weighting matrices used in the cost function (5) are

$$\begin{aligned} Q &= \mathrm{diag}(12, 12, 12,\ 3, 3, 9,\ 0, 0, 1), \\ R &= \mathrm{diag}(2, 2, 2,\ 5). \end{aligned} \tag{8}$$

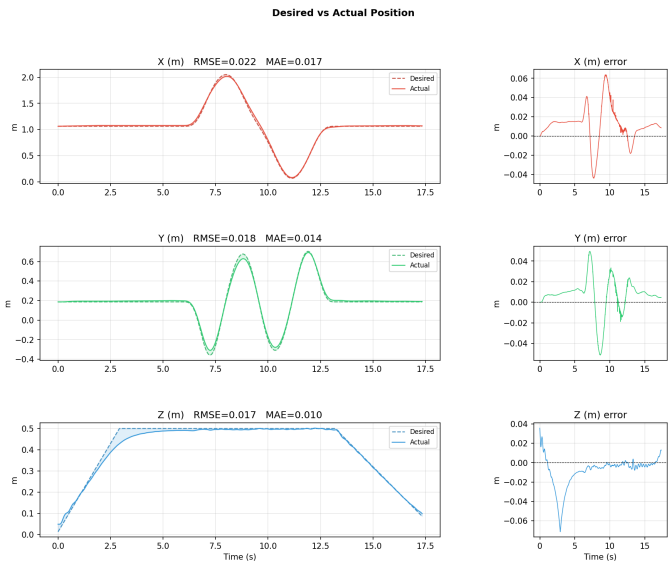
The state penalty matrix Q is structured according to the three subgroups of the state vector. The position weights $(12, 12, 12)$ are the largest in Q, reflecting the primary objective of accurate trajectory tracking. The velocity weights $(3, 3, 9)$ are smaller, but with a heavier penalty on V_z than on V_x and V_y; this asymmetry was found empirically to significantly improve altitude tracking, as the vertical dynamics are more sensitive to velocity mismatch given the direct coupling between $\dot{z}$ and thrust. The attitude weights are $(0, 0, 1)$: roll and pitch receive zero penalty, since penalizing them conflicts with the position objective. To accelerate laterally, the drone must tilt, and any direct penalty on tilt angle would resist the very motion needed to reduce position error. Yaw receives a small penalty to suppress heading drift without constraining the trajectory.

The input penalty matrix R assigns a weight of 2 to the attitude commands $(\phi_{\mathrm{cmd}}, \theta_{\mathrm{cmd}}, \psi_{\mathrm{cmd}})$ and a weight of 5 to the thrust. The light attitude penalty damps unnecessary oscillations in the commanded angles, while the heavier thrust penalty was found to produce smoother thrust profiles throughout the trajectory, reducing high-frequency actuation.

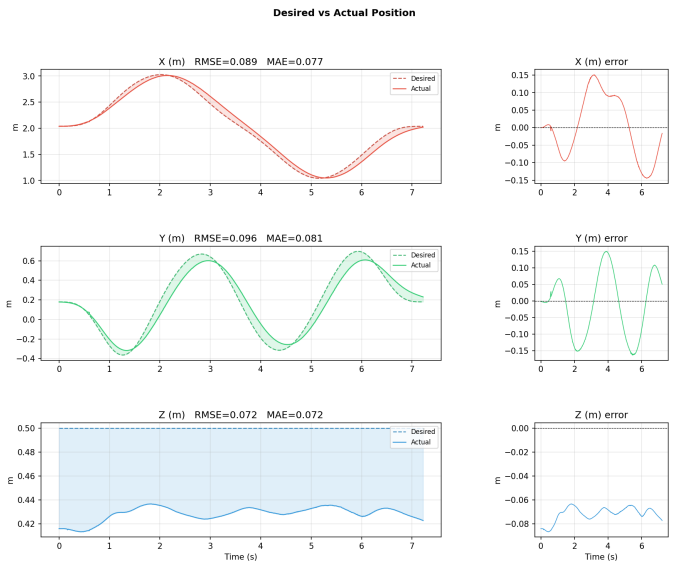
The prediction horizon was set to $T_p = 1.5$ s for trajectory tracking experiments, discretized into $N = 50$ equal shooting intervals, giving a shooting interval of $\Delta t = 30$ ms. A shorter horizon of $T_p = 0.5$ s was used for the disturbance robustness experiment; the effect of horizon length on the robustness-tracking trade-off is discussed further in Section 5.4. The MPC itself runs at 100 Hz.

5.2 Lemniscate Trajectory

The lemniscate trajectory tracking results, presented in Figures 5 and 6, demonstrate that the MPC controller achieved substantially superior tracking performance compared to the Mellinger controller across all three spatial axes. As shown in Figure 5(a), the MPC attained RMSE values of 0.022 m, 0.018 m, and 0.017 m in the X, Y, and Z axes respectively, with corresponding mean absolute errors (MAE) of 0.017 m, 0.014 m, and 0.010 m. The Mellinger controller, shown in Figure 5(b), exhibited considerably higher tracking errors, with RMSE values of 0.089 m, 0.096 m, and 0.072 m across the same axes, representing a degradation in tracking accuracy of approximately $4\times$ relative to the MPC. This discrepancy is further corroborated by the top-down trajectory plots in Figure 6, where the MPC trajectory closely follows the desired lemniscate path with maximum absolute errors of 0.064 m, 0.051 m, and 0.071 m in X, Y, and Z respectively, while the Mellinger controller produced a visibly more deviant path, with maximum absolute errors reaching 0.151 m, 0.163 m, and 0.087 m. Notably, both controllers exhibited their largest instantaneous deviations near the central crossing point of the lemniscate, where the required change in curvature is most abrupt, though the MPC recovered more rapidly and maintained tighter bounds throughout the remainder of the trajectory.

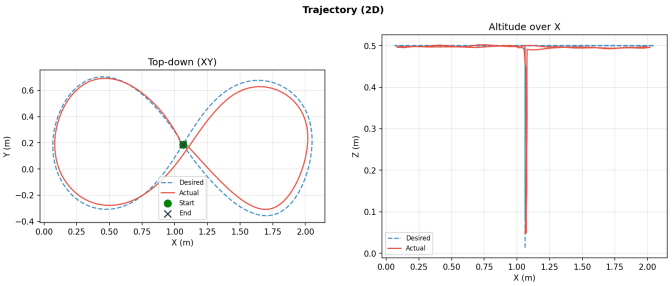


(a) MPC controller: desired vs. actual position and tracking error for X, Y, and Z axes.

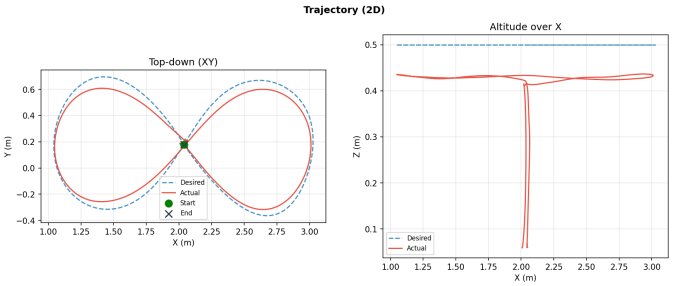


(b) Mellinger controller: desired vs. actual position and tracking error for X, Y, and Z axes.

Figure 5: Position tracking results for the lemniscate trajectory comparing the MPC and Mellinger controllers.



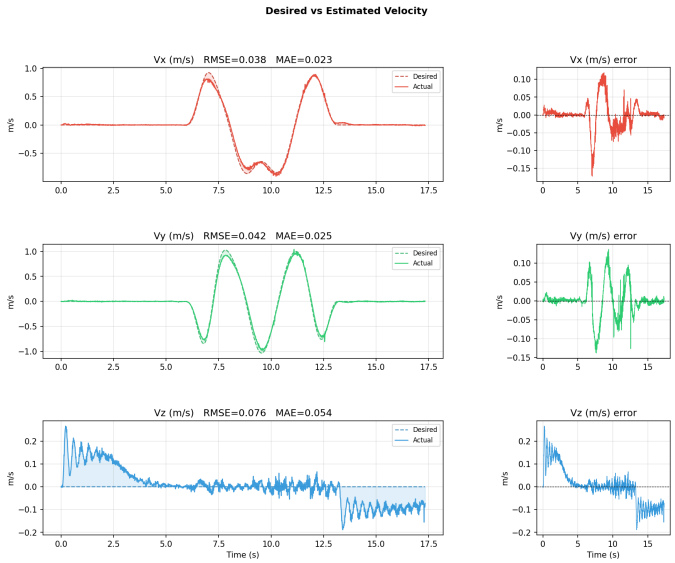
(a) MPC controller: top-down (XY) trajectory and altitude (XZ) profile with mean and maximum absolute errors of 0.017 m & 0.064 m (X), 0.014 m & 0.051 m (Y), and 0.010 m & 0.071 m (Z).



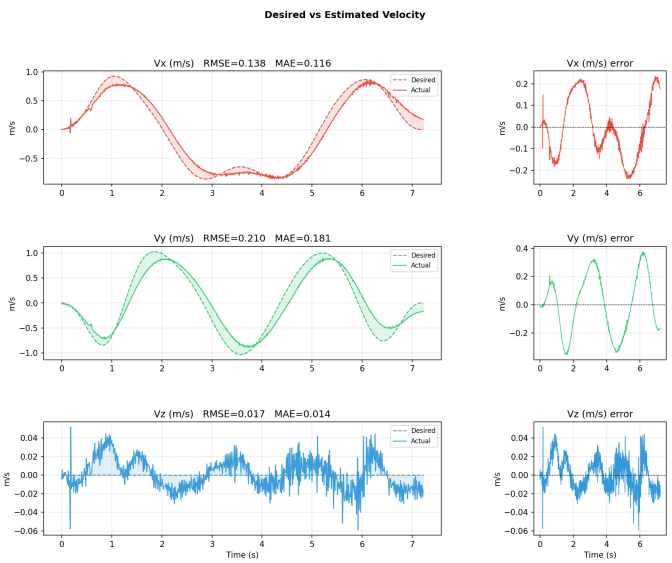
(b) Mellinger controller: top-down (XY) trajectory and altitude (XZ) profile with mean and maximum absolute errors of 0.077 m & 0.151 m (X), 0.082 m & 0.163 m (Y), and 0.072 m & 0.087 m (Z).

Figure 6: Top-down (XY) trajectory and altitude (XZ) profile for the lemniscate tracking task comparing the MPC and Mellinger controllers.

The velocity tracking results, shown in Figure 7, further corroborate the position tracking findings. The MPC controller achieved velocity RMSE values of 0.038 m/s, 0.042 m/s, and 0.076 m/s in V_x, V_y, and V_z respectively, compared to 0.138 m/s, 0.210 m/s, and 0.017 m/s for the Mellinger controller. The MPC maintained tight velocity tracking throughout the lemniscate maneuver, with errors remaining largely bounded within 0.15 m/s even during the high-curvature central crossing. The Mellinger controller, by contrast, exhibited substantially larger lateral velocity errors peaking near 0.4 m/s in V_y, which is consistent with the greater positional drift observed in Figure 5(b). Notably, while the Mellinger controller achieved a lower V_z RMSE (0.017 m/s vs. 0.076 m/s), this is attributable to the near-constant altitude command rather than superior dynamic tracking, and does not offset the persistent negative Z bias visible in its position response. Taken together, the velocity results confirm that the MPC more accurately tracks the required dynamic profile of the trajectory, providing a consistent explanation for its superior positional accuracy.



(a) MPC controller: desired vs. estimated velocity and velocity error for V_x, V_y, and V_z.

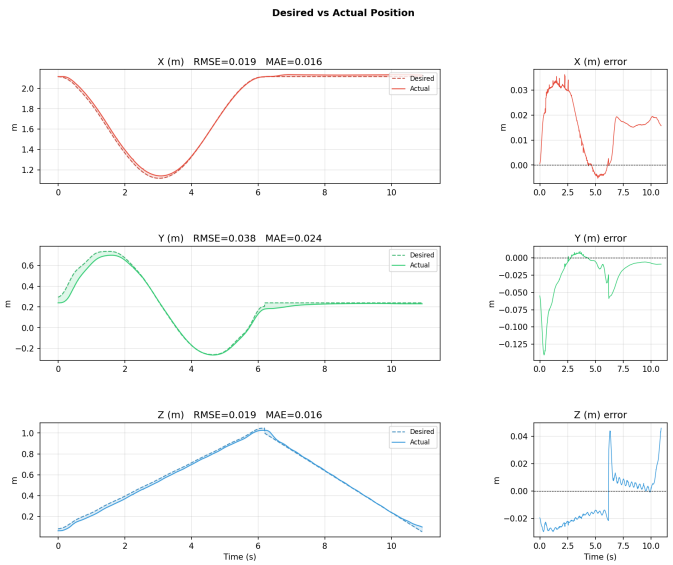


(b) Mellinger controller: desired vs. estimated velocity and velocity error for V_x, V_y, and V_z.

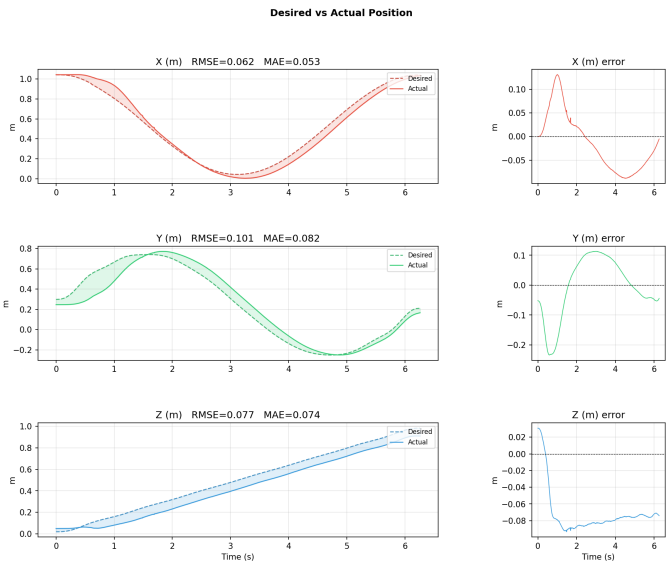
Figure 7: Velocity tracking results for the lemniscate trajectory comparing the MPC and Mellinger controllers.

5.3 Helix comparison

The helix trajectory tracking results, presented in Figures 8 and 9, again demonstrate a clear performance advantage for the MPC controller over the Mellinger controller across all three spatial axes. The MPC achieved RMSE values of 0.019 m, 0.038 m, and 0.019 m in the X, Y, and Z axes respectively, with corresponding mean absolute error values of 0.016 m, 0.024 m, and 0.016 m. The Mellinger controller exhibited substantially higher tracking errors, with RMSE values of 0.062 m, 0.101 m, and 0.077 m, representing a degradation of approximately $3\times$ relative to the MPC. This is further confirmed by the top-down and altitude profiles in Figure 9, where the MPC trajectory closely follows the desired circular helix path with maximum absolute errors of 0.036 m, 0.141 m, and 0.046 m in X, Y, and Z respectively, while the Mellinger controller produced a visibly more deviant path with maximum absolute errors of 0.131 m, 0.234 m, and 0.093 m. Most critically, as visible in the Mellinger top-down and altitude plots of Figure 9, the Mellinger controller failed entirely to execute a controlled transition from the trajectory tracking phase to the landing stage, resulting in an uncontrolled descent and crash. The MPC controller, by contrast, completed the full helix maneuver and transitioned smoothly to landing, highlighting a fundamental robustness advantage of the predictive control formulation for multi-phase trajectory execution.

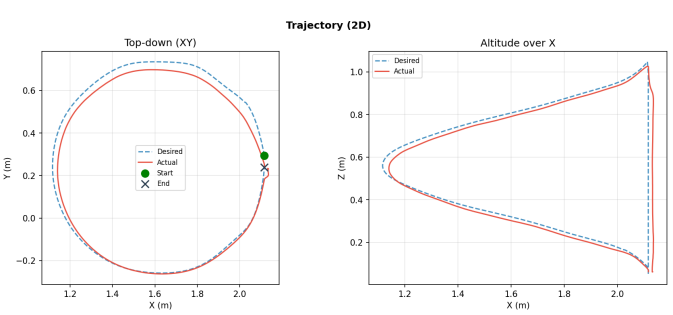


(a) MPC controller: desired vs. actual position and tracking error for X, Y, and Z axes.

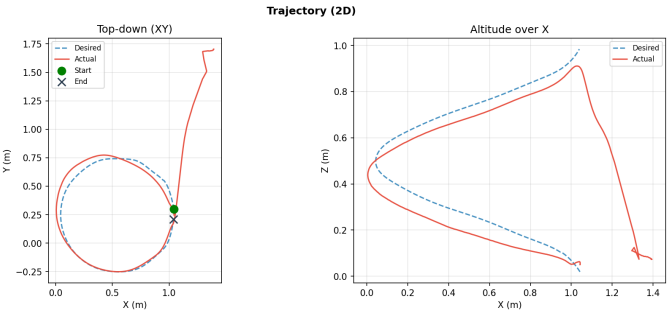


(b) Mellinger controller: desired vs. actual position and tracking error for X, Y, and Z axes.

Figure 8: Position tracking results for the helix trajectory comparing the MPC and Mellinger controllers.



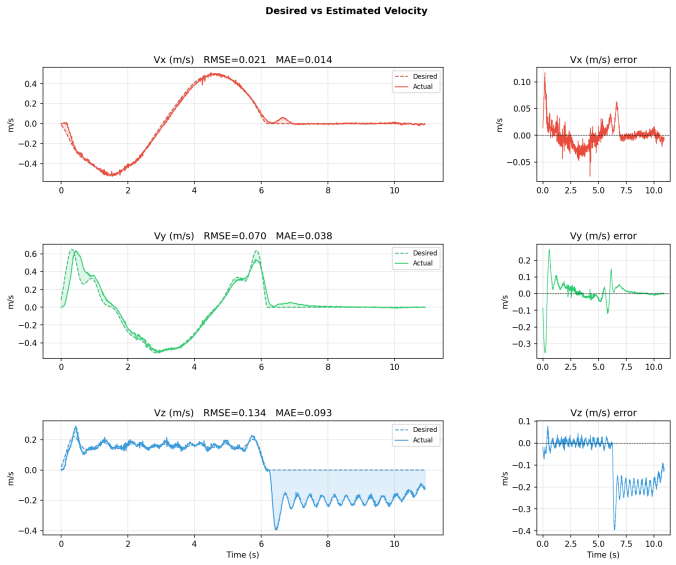
(a) MPC controller: top-down (XY) trajectory and altitude (XZ) profile with mean and maximum absolute errors of 0.016 m & 0.036 m (X), 0.024 m & 0.141 m (Y), and 0.016 m & 0.046 m (Z).



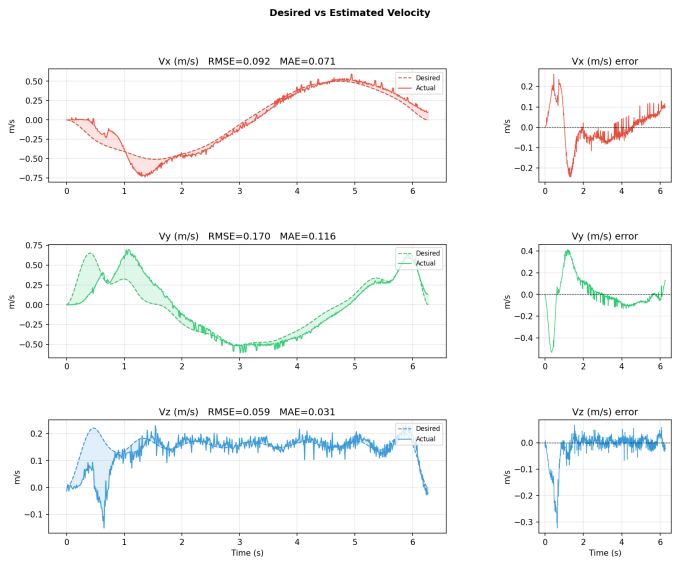
(b) Mellinger controller: top-down (XY) trajectory and altitude (XZ) profile with mean and maximum absolute errors of 0.053 m & 0.131 m (X), 0.082 m & 0.234 m (Y), and 0.074 m & 0.093 m (Z).

Figure 9: Top-down (XY) trajectory and altitude profile for the helix tracking task comparing the MPC and Mellinger controllers.

The velocity tracking results for the helix trajectory, shown in Figure 10, further reinforce the positional findings. The MPC controller achieved velocity RMSE values of 0.021 m/s, 0.070 m/s, and 0.134 m/s in V_x, V_y, and V_z respectively, compared to 0.092 m/s, 0.170 m/s, and 0.059 m/s for the Mellinger controller. The MPC maintained accurate lateral velocity tracking throughout the helix maneuver, with V_x and V_y errors remaining well bounded, consistent with the tight positional tracking observed in Figure 8. The elevated V_z RMSE of 0.134 m/s for the MPC is attributable to the oscillatory behaviour observed after $t \approx 6$ s, corresponding to the landing transition phase, where the quadrotor struggles to smoothly track the rapidly changing vertical velocity command. The Mellinger controller, by contrast, exhibited significantly higher lateral velocity errors, with V_y RMSE reaching 0.170 m/s and peak errors exceeding 0.4 m/s, consistent with its larger positional deviations. Its lower V_z RMSE of 0.059 m/s during the trajectory phase is again misleading, as the controller ultimately failed to execute a controlled landing, as evidenced by the abrupt termination of its trajectory visible in Figure 9b.



(a) MPC controller: desired vs. estimated velocity and velocity error for V_x, V_y, and V_z.



(b) Mellinger controller: desired vs. estimated velocity and velocity error for V_x, V_y, and V_z. Note the abrupt termination of the trajectory due to the uncontrolled descent during the landing transition.

Figure 10: Velocity tracking results for the helix trajectory comparing the MPC and Mellinger controllers.

5.4 Hover robustness comparison

The robustness of both controllers was evaluated under a hover task subject to repeated external disturbances. The resulting trajectories are shown in Figure 11.

The MPC controller, shown in Figure 11(a), exhibited significant transient excursions in both the lateral (XY) and vertical (Z) planes in response to disturbances, with the drone descending to a minimum altitude of 0.05 m before recovering to the hover setpoint. While the recovery time was considerable and the transient deviations were large, the controller was ultimately able to return to the desired position from a severely perturbed state, demonstrating meaningful recovery capability under extreme disturbances.

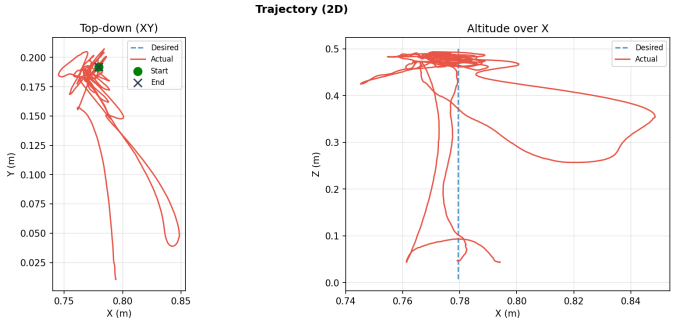
The Mellinger controller, shown in Figure 11(b), performed substantially better in this experiment. It maintained tighter lateral confinement around the hover setpoint, exhibited more consistent roll and pitch angles throughout, and recovered from each disturbance more rapidly. The Mellinger drone never entered the kind of severely off-nominal state that the MPC drone experienced; the disturbances produced only moderate deviations that were corrected promptly without large transient excursions.

This performance difference is structurally expected and reflects a fundamental characteristic of each control paradigm. The Mellinger controller is a reactive geometric controller that computes corrective torques and thrust directly from the instantaneous state error, enabling fast, high-gain disturbance rejection. Its inner attitude loop runs at 250 Hz with proportional-derivative feedback on both attitude error and angular velocity, giving it the ability to suppress impulsive perturbations before they propagate into large positional deviations. The MPC, by contrast, is a predictive controller whose corrective response is shaped by the optimization over the full horizon T_p. Under impulsive disturbances that are not captured in the prediction model, the MPC effectively under-reacts in the short term because the cost function simultaneously penalizes future trajectory deviation, diluting the weight placed on immediate error correction.

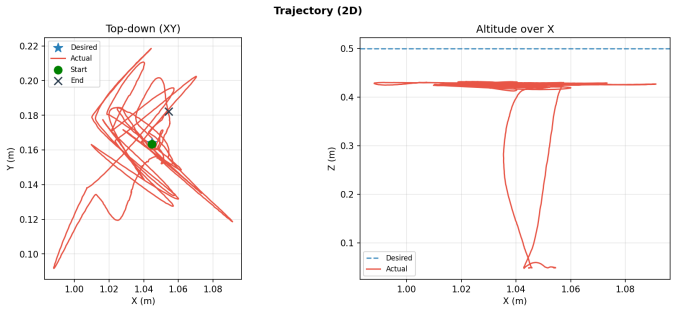
This behavior is consistent with well-established observations in the MPC literature. Longer prediction horizons generally improve nominal trajectory-tracking performance by allowing the controller to better anticipate future system evolution and better approximate the infinite-horizon optimal control problem (Rawlings & Mayne, 2009). From what was observed, as the prediction horizon increases, the effects of model mismatch and disturbances may propagate further into the predicted trajectory, potentially degrading closed-loop performance when uncertainty is significant. Consequently, a substantial body of work on robust MPC has been developed to mitigate the performance degradation and constraint violations that may arise under uncertainty and bounded disturbances (Bemporad & Morari, 1999; Mayne, Seron, & Rakovic, 2005). Furthermore, in the presented experiments, reducing T_p from 1.5 s to 0.5 s improved disturbance rejection during hover, while degrading trajectory-tracking performance on the lemniscate and helix trajectories. This observation is consistent with the practical trade-off between prediction capability and feedback-dominated correction commonly encountered in MPC implementations (Rawlings & Mayne, 2009).

In summary, the two controllers exhibit complementary strengths: the Mellinger controller is the superior choice for disturbance rejection and hover stability, while the MPC excels at precise trajectory tracking and multi-phase maneuver execution. The MPC's

ability to recover from extreme deviations, such as the near-ground excursion observed here, suggests that its predictive structure confers a degree of global recovery capability that a purely reactive controller may lack, even if its predictive structure makes it more susceptible to entering such off-nominal states under impulsive disturbances.



(a) MPC controller: top-down (XY) and altitude (XZ) trajectories showing significant transient excursions followed by recovery to the hover setpoint.



(b) Mellinger controller: top-down (XY) and altitude (XZ) trajectories showing tighter lateral confinement and more rapid recovery from disturbances compared to the MPC controller.

Figure 11: Robustness evaluation during a hover task under external disturbances comparing the MPC and Mellinger controllers.

5.5 Lemniscate reliability

To assess controller reliability over extended flight durations, the lemniscate trajectory was executed five times consecutively without intermediate landing. The resulting trajectory-tracking performance is summarized in Figures 12 and 13.

Figure 12 confirms that the desired path is followed adequately across all five repetitions. However, a systematic offset between the desired and actual trajectory is apparent, which is more pronounced here than in the individual single-run experiments. This is reflected quantitatively in the position RMSEs of 0.075 m, 0.064 m, and 0.043 m in X, Y, and Z respectively—considerably higher than the single-run values of 0.022 m, 0.018 m, and 0.017 m. A persistent altitude offset of approximately 0.042 m is also visible throughout the flight.

The cause of this degradation is attributed to a systematic change in the experimental setup, most likely mechanical wear and tare on the drone or the MoCap marker frame, or drift in the PWM-to-thrust calibration curve, rather than to any property of the consecutive trajectory execution itself. This hypothesis is supported by the observation that the offset persisted when the individual trajectory experiments were subsequently repeated. Unfortunately, recalibration was not possible within the project timeline.

Importantly, despite the systematic offset, the relative tracking quality remains highly consistent across all five runs, as is evident from the position error profiles in Figure 13a: the error magnitude and character are preserved throughout the full flight duration. This demonstrates that the MPC controller maintains stable, repeatable performance over extended operation without exhibiting drift or degradation in its control behavior.

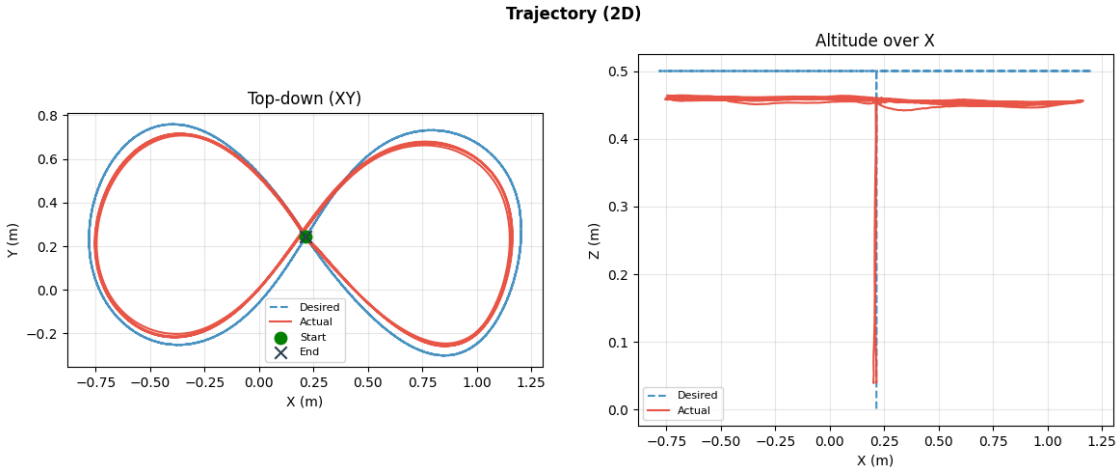
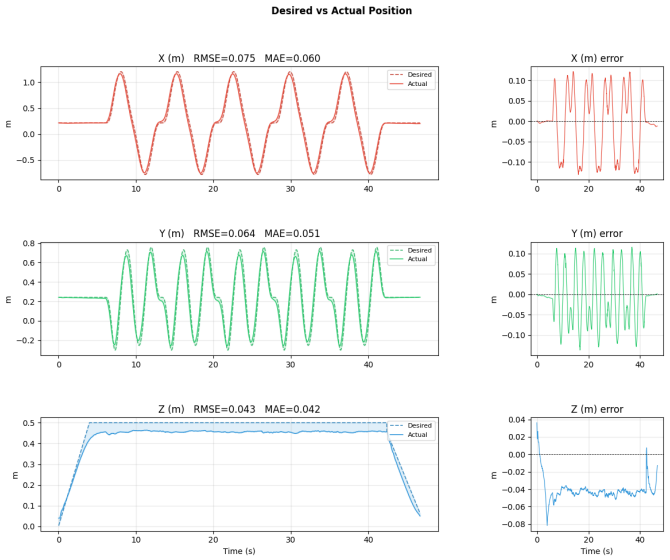
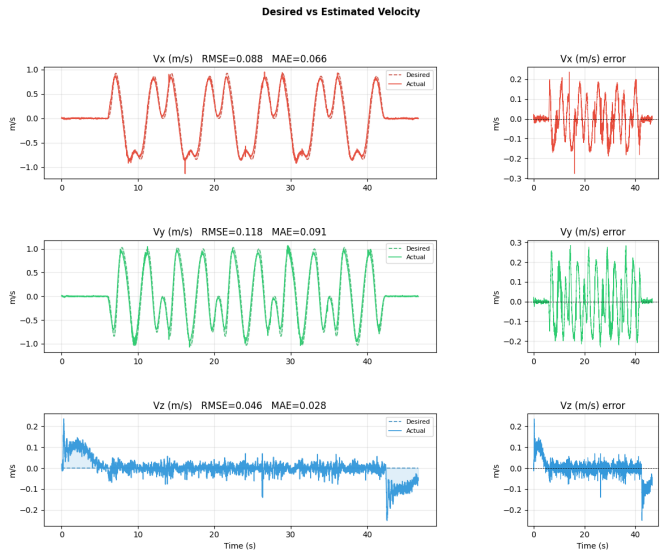


Figure 12: MPC controller: top-down (XY) trajectory and altitude (XZ) profile with mean and maximum absolute errors of 0.060 m & 0.13 m (X), 0.051 m & 0.14 m (Y), and 0.042 m & 0.08 m (Z).



(a) MPC controller: desired vs. estimated position and position error for X, Y, and Z axes.



(b) MPC controller: desired vs. estimated velocity and velocity error for V_x, V_y, and V_z.

Figure 13: Position and velocity tracking results of the MPC controller for the lemniscate trajectory performed 5 times consecutively.

6 Future work

This project shows the viability of Model Predictive Controllers for accurate trajectory tracking on the Crazyflie 2.1 nano-quadrotor platform. The immediate next step could be the transition from trajectory tracking MPC to path following MPC, as implemented in (Leprich et al., 2026). In trajectory tracking, the controller is required to be at a specific position at a specific time, meaning tracking error accumulates whenever the vehicle falls behind or ahead of the reference schedule. Path following MPC decouples spatial accuracy from temporal progress by treating the arc-length parameter along the path as a free variable optimized online. This allows the vehicle to slow down through high-curvature sections and accelerate along straight segments, improving spatial accuracy without incurring the timing penalties inherent to trajectory tracking. For practical applications such as inspection or swarm coordination, where adherence to the geometric path matters more than strict timing, this reformulation offers a direct performance benefit.

Other avenues involve more data-driven approaches. This work employed a simplified cascaded model in which the closed-loop roll and pitch dynamics are approximated as first-order lags (linear rotational dynamics). This approximation is accurate in the low-to-moderate agility regime, where attitudes remain small, accelerations are gentle, and the dominant unmodeled effects, motor dynamics, propeller aerodynamics, and blade flapping forces, are negligible. However, the approximation degrades in several practically relevant scenarios. At high speeds or during aggressive maneuvers, large attitude angles may invalidate the first order rotational model, and aerodynamic effects such as blade flapping and induced drag become significant. Similarly, when operating near surfaces or other vehicles, ground and ceiling effects alter the effective thrust coefficient in ways not captured by the static PWM-to-thrust calibration. External turbulence introduces disturbances that are neither predictable nor representable by the nominal model, causing the MPC to plan over an increasingly inaccurate prediction. In all of these regimes, the mismatch between the prediction model and the true plant grows, and the MPC loses the ability to anticipate and preempt the resulting deviations. This can be accounted for by using data collected from the real system to model the unaccounted-for non-linear dynamics in the system. This can be done by using Gaussian Processes to learn residuals between the nominal model and the true plant dynamics from flight data. The learned GP can then be incorporated directly into the MPC prediction model, providing both improved mean predictions and uncertainty quantification. Crucially, the predictive variance of the GP can be propagated through the horizon to derive probabilistic constraint tightening, yielding a controller that provides formal guarantees of constraint satisfaction with high probability even in the presence of unmodeled dynamics. This direction is particularly relevant for safe operation in uncertain or cluttered environments. This can also be approached by substituting the physical model with a Neural Network and learning the dynamics entirely from data. These approaches would require large amounts of data, which could prove to be a bottleneck to their feasibility, since collecting sufficiently rich data is not trivial.

While obstacle-avoidance constraints can, in principle, be embedded directly in the OCP, doing so typically introduces nonconvexity and significantly increases solve time. An alternative is to implement a Control Barrier Function (CBF) safety filter as a lightweight QP running downstream of the MPC: the MPC generates the nominal trajectory-tracking command, and the CBF-QP

perturbs it minimally to enforce safety constraints in real time (Ames et al., 2019). This separation preserves the 10 ms solve budget of the MPC while adding a computationally cheap safety layer, making it well suited to the Crazyflie platform and could be easily implemented in the current architecture since it requires little to no changes to the controller.

The Crazyswarm2 middleware natively supports multi-Crazyflie operation. Extending the current architecture to a swarm setting, where each agent runs an individual MPC and agents must avoid collision with one another, is a natural next step. Distributed MPC formulations, in which agents share predicted trajectories to enforce inter-agent constraints, would allow the collision avoidance problem to be solved cooperatively without a centralized coordinator.

The current off-board architecture is limited by the Crazyradio communication link, which introduces latency and is susceptible to packet loss. Porting the MPC solver to a companion computer mounted on a larger quadrotor platform, or leveraging embedded solver toolchains such as acados with generated C code, would eliminate this bottleneck and enable fully autonomous, infrastructure-independent operation. This has already been achieved before on the same platform (Nguyen, Schoedel, Alavilli, Plancher, & Manchester, 2024), and thus its feasibility has been proven.

7 Conclusion

This work presented the design, implementation, and experimental validation of an off-board Model Predictive Controller for trajectory tracking on the Crazyflie 2.1 nano-quadrotor. The controller employs a cascaded Separated Guidance and Control architecture, delegating fast attitude stabilization to the onboard PID firmware while the MPC outer loop solves a 9-state constrained Optimal Control Problem at 100 Hz using acados and CasADi.

Two practical engineering contributions supported the implementation: a hybrid velocity estimator combining MoCap finite differencing with onboard EKF fallback for outlier rejection, and an improvised thrust calibration stand that yielded a highly linear PWM-to-thrust mapping ($R^2 = 0.9989$) without specialist equipment.

Experimental results across lemniscate and helical trajectories demonstrated a consistent 3–4$\times$ reduction in position RMSE relative to the Crazyflie's built-in Mellinger controller, with the MPC achieving sub-centimeter mean tracking errors on the lemniscate. On the helix trajectory, the Mellinger controller failed to execute a controlled landing transition, whereas the MPC completed the full maneuver and landed safely, underscoring the robustness advantage of the predictive formulation for multi-phase trajectories. The five-lap lemniscate reliability experiment confirmed that tracking quality is maintained over extended flight durations without degradation.

The primary limitation of the current approach is its reliance on a simplified prediction model, which reduces disturbance rejection performance relative to a reactive controller such as Mellinger under impulsive perturbations. The horizon-length trade-off between tracking accuracy and disturbance robustness was identified as a key tuning consideration. A further practical limitation is the off-board architecture, which introduces communication latency and packet-loss sensitivity.

Taken together, these results establish that MPC is a viable and high-performing control strategy for nano-quadrotor trajectory tracking, offering constraint awareness, multi-phase robustness, and a natural pathway toward safe learning-based and obstacle-aware extensions that reactive controllers cannot easily accommodate.

References

Ames, A. D., Coogan, S., Egerstedt, M., Notomista, G., Sreenath, K., & Tabuada, P. (2019, June). Control barrier functions: Theory and applications. In *2019 18th european control conference (ecc)* (pp. 3420–3431). Napoli, Italy.

Andersson, J., Gillis, J., Horn, G., Rawlings, J., & Diehl, M. (2018, 07). Casadi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, *11*. doi: 10.1007/s12532-018-0139-4

Bemporad, A., & Morari, M. (1999). Robust model predictive control: A survey. In *Robustness in identification and control* (pp. 207–226). Springer.

Bitcraze. (n.d.). *Crazyflie 2.1.* https://www.bitcraze.io/products/old-products/crazyflie-2-1/. Retrieved from https://www.bitcraze.io/products/old-products/crazyflie-2-1/

Bitcraze AB. (n.d.). *Crazyflie firmware documentation: Controllers.* Retrieved from https://www.bitcraze.io/documentation/repository/crazyflie-firmware/master/functional-areas/sensor-to-control/controllers/

Bitcraze AB. (2026). *Pwm to thrust.* Retrieved 2026-06-05, from https://www.bitcraze.io/documentation/repository/crazyflie-firmware/master/functional-areas/pwm-to-thrust/ (Crazyflie Firmware Documentation)

Kamel, M. S., Stastny, T., Alexis, K., & Siegwart, R. (2017, 05). Model predictive control for trajectory tracking of unmanned aerial vehicles using robot operating system.. doi: 10.1007/978-3-319-54927-9_1

Leprich, D., Rosenfelder, M., Hermle, M., Chen, J., & Eberhard, P. (2026, jan). Model predictive path-following control for a quadrotor. *Proceedings in Applied Mathematics and Mechanics*, *26*(1). Retrieved from http://dx.doi.org/10.1002/pamm.70056 doi: 10.1002/pamm.70056

Llanes, C., Kakish, Z., Williams, K., & Coogan, S. (2024). Crazysim: A software-in-the-loop simulator for the crazyflie nano quadrotor. In *2024 ieee international conference on robotics and automation (icra)* (p. 12248-12254). doi: 10.1109/ICRA57147.2024.10610906

Mayne, D. Q., Seron, M. M., & Rakovic, S. V. (2005). Robust model predictive control of constrained linear systems with bounded disturbances. *Automatica*, *41*(2), 219–224.

Mellinger, D., & Kumar, V. (2011). Minimum snap trajectory generation and control for quadrotors. In *Proceedings of the ieee international conference on robotics and automation (icra)* (pp. 2520–2525). doi: 10.1109/ICRA.2011.5980409

Nguyen, K., Schoedel, S., Alavilli, A., Plancher, B., & Manchester, Z. (2024). Tinympc: Model-predictive control on resource-constrained microcontrollers. In *Ieee international conference on robotics and automation (icra).*

Preiss*, J. A., Hönig*, W., Sukhatme, G. S., & Ayanian, N. (2017). Crazyswarm: A large nano-quadcopter swarm. In *IEEE international conference on robotics and automation (ICRA)* (pp. 3299–3304). IEEE. Retrieved from https://doi.org/10.1109/ICRA.2017.7989376 (Software available at https://github.com/USC-ACTLab/crazyswarm) doi: 10.1109/ICRA.2017.7989376

Rawlings, J. B., & Mayne, D. Q. (2009). *Model predictive control: Theory and design.* Nob Hill Publishing.

Richter, C., Bry, A., & Roy, N. (2016). Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments. In *Robotics research* (pp. 649–666). Springer.

Verschueren, R., Frison, G., Kouzoupis, D., Frey, J., van Duijkeren, N., Zanelli, A., … Diehl, M. (2021, 10). acados—a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, *14*, 1-37. doi: 10.1007/s12532-021-00208-8